天津工業大學
TIANGONG UNIVERSITY

毕业设计（论文）

基于 Structured Streaming 的
实时舆情分析与热度趋势监测系统设计

学生姓名： 高楠

学　院： 计算机科学与技术学院

专业班级： 大数据 2201

指导教师： 陈志

完成日期： 2026 年 6 月

摘 要

近年来，随着微博、抖音、小红书、B 站、知乎等社交媒体平台的快速兴起，用户生成内容（User Generated Content，UGC）呈现指数级增长态势，舆情信息的实时感知与分析已成为政府公共治理、企业品牌维护、社会应急管理等领域的核心技术需求。然而，传统基于批处理架构的舆情分析系统普遍存在处理延迟高、数据时效性差、无法快速响应突发舆情事件等固有局限，难以满足当前高频、海量、多源异构数据环境下的实时分析要求。

本文针对上述问题，设计并实现了一套基于 Apache Spark Structured Streaming 的实时舆情分析与热度趋势监测系统。系统采用 Kafka 作为高吞吐消息队列，以 60 秒为一个微批次触发周期持续消费多平台社交媒体评论数据流；在流处理层集成阿里巴巴达摩院 StructBERT 中文预训练语言模型，实现对评论文本的情感极性分类（正向/负向/中性），并据此构建综合考量点赞数、转发数与情感极性的多维热度评分模型；存储层采用 Delta Lake 数据湖仓一体方案，设计 Bronze-Silver-Gold 三层渐进式数据模型，分别承担原始数据落盘、清洗去重以及多粒度聚合统计的职责；服务层基于 FastAPI 构建 RESTful 数据接口，前端采用 Vue 3 与 ECharts 实现多话题实时监测可视化大屏；系统还内置了基于规则的负面舆情告警机制，当检测到特定话题在某小时内负面评论占比超过阈值时，自动触发邮件告警并持久化预警记录。

本系统在阿里云 Linux 服务器环境下完成部署与验证，实验结果表明，系统能够在 60 秒处理周期内完成对多话题评论流的接收、情感推理、热度计算与数据入湖，端到端延迟满足准实时监测需求，各模块功能指标均达到设计预期。

关键词：流式计算；Spark Structured Streaming；舆情分析；情感分类；StructBERT；Delta Lake；热度模型；实时告警

Abstract

In recent years, with the rapid rise of social media platforms such as Weibo, Douyin, Xiaohongshu, Bilibili, and Zhihu, user-generated content (UGC) has grown exponentially. The real-time perception and analysis of public opinion information has become a core technological requirement in government governance, corporate brand management, and social emergency response. However, traditional batch-processing-based public opinion analysis systems generally suffer from high processing latency, poor data timeliness, and inability to respond quickly to emergent public opinion events, making them inadequate for the demands of high-frequency, massive, and heterogeneous streaming data environments.

In response to the above challenges, this paper designs and implements a real-time public opinion analysis and heat trend monitoring system based on Apache Spark Structured Streaming. The system uses Apache Kafka as a high-throughput message queue, triggering micro-batch processing every 60 seconds to continuously consume multi-platform social media comment streams. At the stream processing layer, the system integrates the StructBERT Chinese pre-trained language model from Alibaba DAMO Academy to perform sentiment polarity classification (positive/negative/neutral) on comment texts, and constructs a multi-dimensional heat scoring model that incorporates like counts, repost counts, and sentiment polarity. The storage layer adopts the Delta Lake lakehouse architecture, implementing a three-tier progressive data model (Bronze-Silver-Gold) responsible for raw data ingestion, deduplication and cleansing, and multi-granularity aggregation statistics respectively. The service layer exposes RESTful APIs via FastAPI, while the frontend employs Vue 3 and ECharts to deliver a real-time multi-topic monitoring dashboard. The system also incorporates a rule-based negative sentiment alerting mechanism that automatically triggers email notifications and persists alert records when the ratio of negative comments for a given topic in a specific hour exceeds the configured threshold.

The system was deployed and validated on an Alibaba Cloud Linux server. Experimental results demonstrate that the system can complete the reception, sentiment in-

ference, heat computation, and data lakehouse ingestion of multi-topic comment streams within a 60-second processing cycle, with end-to-end latency satisfying the requirements of near-real-time monitoring. All module functional metrics meet the design expectations.

Keywords: Stream Computing; Spark Structured Streaming; Public Opinion Analysis; Sentiment Classification; StructBERT; Delta Lake; Heat Scoring Model; Real-Time Alerting

目录

摘要 ..... **I**

Abstract ..... **II**

第一章　绪论 **1**

1.1　研究背景与意义 1

1.1.1　社交媒体时代的舆情挑战 1

1.1.2　传统舆情系统的局限性 1

1.1.3　研究意义 2

1.2　国内外研究现状 2

1.2.1　流式计算框架研究现状 2

1.2.2　情感分析技术研究现状 2

1.2.3　数据湖仓技术研究现状 3

1.2.4　现有研究的不足与本文的切入点 3

1.3　研究内容与主要工作 3

1.4　论文组织结构 4

第二章　相关技术与理论基础 **5**

2.1　Apache Kafka 消息队列 5

2.1.1　Kafka 基本架构 5

2.1.2　Kafka 的核心特性 5

2.1.3　本系统中 Kafka 的应用 5

2.2　Apache Spark Structured Streaming 6

2.2.1　Structured Streaming 的设计哲学 6

2.2.2　微批次处理模型 6

2.2.3　`foreachBatch` 接口 6

2.2.4　Checkpoint 与容错保障 6

2.3　Delta Lake 数据湖仓架构 7

2.3.1　Delta Lake 概述 7

2.3.2　核心特性 7

2.3.3 三层数据架构设计思想（Bronze-Silver-Gold） 7
2.4 StructBERT 中文情感分析模型 8
2.4.1 BERT 预训练语言模型基础 8
2.4.2 StructBERT 的结构增强 8
2.4.3 本系统中的情感分析设计 9
2.5 FastAPI 后端框架 9
2.6 Vue 3 与 ECharts 可视化技术 10
2.6.1 Vue 3 前端框架 10
2.6.2 Apache ECharts 数据可视化 10
2.6.3 技术栈选型小结 10
第三章 系统需求分析与总体设计 12
3.1 功能需求分析 12
3.1.1 多平台数据采集需求 12
3.1.2 实时流数据接入需求 12
3.1.3 情感分析与热度计算需求 12
3.1.4 数据存储与管理需求 12
3.1.5 负面舆情告警需求 13
3.1.6 可视化展示需求 13
3.2 非功能需求分析 13
3.2.1 实时性需求 13
3.2.2 可靠性需求 13
3.2.3 可扩展性需求 13
3.2.4 可维护性需求 14
3.3 系统总体架构设计 14
3.3.1 架构分层说明 14
3.3.2 技术架构关键决策 14
3.4 数据流设计 16
3.4.1 数据采集阶段数据流 16
3.4.2 流处理阶段数据流 16
3.4.3 数据服务阶段数据流 17
3.5 模块划分 17
3.6 关键接口定义 18

3.6.1 Kafka 消息接口 18
3.6.2 REST API 接口规范 18
3.6.3 告警触发规则接口 18
第四章 系统详细设计与实现 19
4.1 多平台社交媒体数据采集模块 19
4.1.1 模块设计目标 19
4.1.2 爬虫工厂模式 19
4.1.3 CDP 反检测机制 20
4.1.4 轮询采集模式 20
4.1.5 JSONL 输出设计 20
4.2 Kafka 消费管道与数据解析模块 20
4.2.1 Kafka 读取配置 20
4.2.2 JSON 解析与字段校验 21
4.3 StructBERT 情感分析服务模块 21
4.3.1 单例服务设计 21
4.3.2 批量推理流程 22
4.3.3 标签归一化与中性判定 22
4.3.4 异常降级策略 22
4.4 热度计算与风险评估模块 23
4.4.1 热度评分模型 23
4.4.2 风险等级划定规则 24
4.5 Delta Lake 三层数据湖写入模块 24
4.5.1 Bronze 层：原始数据落盘 25
4.5.2 Silver 层：去重精炼写入 25
4.5.3 Gold 层：多粒度聚合指标 25
4.5.4 数据一致性保障 26
4.6 负面舆情告警模块 27
4.6.1 告警触发逻辑 27
4.6.2 告警邮件设计 27
4.6.3 告警配置管理 27
4.7 REST API 数据服务模块 27
4.7.1 数据读取策略 27

4.7.2 数据序列化处理 27
4.7.3 核心接口实现 28
4.8 前端可视化模块 28
4.8.1 整体布局设计 28
4.8.2 数据获取与状态管理 28
4.8.3 ECharts 图表渲染 29
4.8.4 响应式适配 29
4.9 系统部署配置 29
4.9.1 云服务器端部署 29
4.9.2 Spark 提交配置 29
第五章 系统测试与实验分析 31
5.1 测试环境 31
5.1.1 服务器端环境 31
5.1.2 本地开发端环境 31
5.1.3 测试数据集说明 31
5.2 功能测试 32
5.2.1 数据采集模块功能测试 32
5.2.2 流处理管道功能测试 32
5.2.3 情感分析模块功能测试 33
5.2.4 告警模块功能测试 33
5.2.5 前端可视化功能测试 34
5.3 性能测试 34
5.3.1 端到端延迟测试 34
5.3.2 情感推理吞吐量测试 34
5.3.3 API 接口响应时间测试 34
5.3.4 Delta Lake 存储开销分析 35
5.4 实验结果综合分析 35
5.4.1 情感分析准确性分析 35
5.4.2 热度模型有效性分析 36
5.4.3 负面占比阈值合理性分析 36
5.4.4 系统局限性客观讨论 36

第六章 总结与展望 38
6.1 工作总结 38
6.2 不足之处 39
6.3 未来展望 39
致谢 41
参考文献 42

第一章　绪论

1.1　研究背景与意义

1.1.1　社交媒体时代的舆情挑战

随着移动互联网的高速普及与智能手机的全面渗透，微博、B 站、抖音、小红书、知乎等社交媒体平台已成为公众发表观点、传播信息的核心渠道。据中国互联网络信息中心（CNNIC）2025 年统计报告，中国网民规模已突破 11 亿人，其中短视频与社交媒体用户占比超过 90% [1]。海量用户在这些平台上每天产生数以亿计的评论、弹幕与转发，形成极具参考价值的实时公众情绪镜像。

然而，这一信息富矿同时带来了严峻的舆情管理挑战。其一，数据规模庞大，人工审核难以覆盖；其二，信息传播速度极快，负面舆情往往在数小时内扩散至数百万用户；其三，话题情绪复杂多变，具有强烈的事件驱动特征。针对上述挑战，构建一套能够对多平台评论数据进行实时采集、自动分析与主动预警的系统，具有重要的学术价值与现实意义 [2]。

1.1.2　传统舆情系统的局限性

现有商用舆情监测产品（如鹰眼速读网、清博大数据等）通常采用批处理架构：系统以小时或天为粒度定期从数据平台拉取快照，分析完成后生成静态报告。这一模式存在以下固有缺陷：

1. **时效性不足：**批处理周期通常以小时计，难以在舆情爆发初期即发出预警，错失最佳响应窗口。
2. **情感分析粗糙：**多数产品采用基于关键词词典的规则匹配方法，难以理解语境、处理隐性负面情绪及反讽表达。
3. **数据孤岛问题：**不同平台的数据采集、分析与展示模块相互独立，全链路数据难以整合追溯。
4. **可扩展性受限：**新增监测话题或平台往往需要复杂的系统配置，缺乏灵活性。

1.1.3　研究意义

针对上述痛点，本文设计并实现一套基于 Spark Structured Streaming 的实时舆情分析与热度趋势监测系统。该系统以预训练大语言模型 StructBERT 替代词典规则进行情感判定，以 Delta Lake 湖仓架构替代传统关系型数据库，实现流批一体的数据存储与查询。系统的学术意义在于：（1）探索了预训练中文 NLP 模型在低延迟流处理场景下的工程化部署路径；（2）验证了 Delta Lake Medallion 架构在准实时舆情分析系统中的适用性与可靠性；（3）提出了一种融合用户互动行为与情感极性的多维度热度评分模型。

1.2　国内外研究现状

1.2.1　流式计算框架研究现状

流式数据处理领域经历了从第一代（Apache Storm [3]）的纯流式处理，到第二代（Spark Streaming [4]）的微批次处理，再到第三代（Apache Flink [5]、Spark Structured Streaming [6]）的统一流批处理的演进历程。

Zaharia 等人 [4] 于 2012 年提出的离散化流（DStream）模型将流处理问题转化为一系列小批量 RDD 操作，奠定了 Spark Streaming 的理论基础。Armbrust 等人 [6] 在 2018 年进一步提出 Structured Streaming，将流视为无界关系表，引入 Event-Time 语义与 Watermark 机制，显著提升了处理语义的严谨性与 API 的易用性。相较于 Flink 的原生毫秒级流处理，Structured Streaming 在与 Python 生态（NumPy、PyTorch 等）的集成方面具有更成熟的工程支持，更适合本文所需的”流式调度 + 批量 NLP 推理”混合场景。

1.2.2　情感分析技术研究现状

中文情感分析技术经历了词典规则方法（HowNet 情感词典 [7]）、传统机器学习（SVM、朴素贝叶斯）到深度学习（LSTM [8]、BERT [9]）的三个发展阶段。

BERT [9] 的提出引发了预训练语言模型在情感分析任务上的研究热潮。针对中文特点，阿里达摩院提出的 StructBERT [10] 在 BERT 基础上引入词序重建与句子顺序预测两类新的预训练任务，显著提升了模型对中文词语结构的理解能力，在 GLUE 基准的多项子任务上超越了 BERT-Large。本系统选用 StructBERT 中文情感分类专项模型（`iic/nlp_structbert_sentiment-classification_chinese-base`），实验表明其在社交媒体评论上总体准确率可达 83.5%（详见第 5.3.2 节）。

1.2.3　数据湖仓技术研究现状

Delta Lake [11] 由 Databricks 团队于 2020 年开源，是基于 Parquet 文件格式构建的开放 ACID 存储层，通过事务日志（Transaction Log）在对象存储上实现 Serializable 级别的隔离性。与传统数据仓库（Hive/Presto）相比，Delta Lake 的核心优势在于：（1）流批统一写入，Structured Streaming 的 `writeStream` 可直接写入 Delta 表；（2）Time Travel 支持按版本或时间戳查询历史快照；（3）Schema 演化，允许在不中断下游查询的情况下新增字段。

Medallion 架构（Bronze-Silver-Gold）是 Databricks 推荐的 Delta Lake 数据组织模式 [12]，已在字节跳动、阿里巴巴等国内大型互联网企业的数据湖建设实践中得到广泛应用。本系统将该架构应用于准实时舆情分析场景，是对其在小规模单机部署条件下适用性的探索性验证。

1.2.4　现有研究的不足与本文的切入点

综合梳理现有工作，可以发现以下不足：其一，多数基于 Spark 的舆情分析研究局限于离线批处理，缺乏对准实时流处理场景的系统性验证；其二，将预训练大模型（BERT 类）嵌入 Structured Streaming `foreachBatch` 回调的工程化方案鲜有公开详述；其三，Delta Lake Medallion 架构在中文社交媒体舆情场景中的完整实践案例较少。本文针对上述不足，设计并实现完整的端到端系统，对上述三个方面均作出了有价值的工程实践贡献。

1.3　研究内容与主要工作

本文的主要研究内容与工作成果概括如下：

1. **多平台社交媒体爬虫工程化实现。**基于 MediaCrawler 框架与 CDP 接管机制，实现了支持微博、B 站、抖音、小红书、知乎、快手、贴吧七平台的关键词评论采集系统，并设计了 JSONL 格式的低耦合 Kafka 数据输出接口。
2. **Spark Structured Streaming 实时消费管道。**设计了基于 60 秒微批次触发的 Kafka-Spark 消费管道，实现了 JSON Schema 解析、字段校验与字段异常容错处理。
3. **StructBERT 情感分析服务的流处理集成。**提出了懒初始化单例模式 + 批量推理 + 异常降级三层设计方案，将 StructBERT Pipeline 嵌入 `foreachBatch` 回调，实现单例复用、批量高效推理。

4. **多维度热度评分模型设计与实现。**提出了融合点赞数、转发数与情感极性加成的热度评分公式，并设计了基于热度分数与情感标签组合的三级风险评估机制。
5. **Delta Lake Bronze-Silver-Gold 三层湖仓架构实现。**设计并实现了原始数据追加（Bronze）、去重精炼 Merge（Silver）与多粒度聚合全量重算（Gold）三层写入方案，保障了数据的 ACID 一致性与多时间粒度查询能力。
6. **FastAPI 数据服务与 Vue 3 可视化大屏。**设计了 9 个 RESTful 端点，基于 `deltalake` Python 库实现了无 JVM 的 Delta Lake 直读方案；前端基于 Vue 3 Composition API 与 ECharts 6 实现了多话题切换、多粒度趋势图与告警面板的交互式可视化大屏。

1.4　论文组织结构

本文共分六章，各章内容安排如下：

第一章（绪论）：介绍研究背景、意义及国内外研究现状，阐述本文的研究切入点与主要工作。

第二章（相关技术与理论基础）：系统介绍本文所涉及的核心技术，包括 Apache Kafka、Spark Structured Streaming、Delta Lake、StructBERT、FastAPI 及 Vue 3/ECharts。

第三章（系统需求分析与总体设计）：在需求分析基础上，给出系统五层架构设计、数据流设计、模块划分及关键接口定义。

第四章（系统详细设计与实现）：按模块详细阐述各子系统的设计方案与关键代码实现，重点介绍爬虫模块、流处理管道、情感分析服务、热度计算模块与 Delta Lake 写入逻辑。

第五章（系统测试与实验分析）：描述测试环境与数据集，通过功能测试与性能测试对系统各模块进行验证，并对情感分析准确率、热度模型有效性等核心指标进行综合分析。

第六章（总结与展望）：总结全文工作，客观指出系统的局限性，并对未来改进方向进行展望。

第二章　相关技术与理论基础

2.1　Apache Kafka 消息队列

2.1.1　Kafka 基本架构

Apache Kafka [13] 是由 LinkedIn 公司开发、后捐献给 Apache 基金会的分布式流式消息系统，基于发布/订阅（Publish/Subscribe）模式设计。Kafka 的核心概念包括：

- **Broker**：Kafka 集群的服务节点，负责消息的持久化存储与转发。
- **Topic**：消息的逻辑分类，每个 Topic 可配置多个 Partition 实现并行读写。
- **Producer**：向指定 Topic 写入消息的客户端进程。
- **Consumer Group**：消费者组内各实例分配不同 Partition，保证每条消息在组内仅被消费一次，实现水平扩展。
- **Offset**：Consumer 在 Partition 中消费进度的游标，由 Consumer 自主管理，支持精确一次（Exactly-Once）或至少一次（At-Least-Once）语义。

2.1.2　Kafka 的核心特性

Kafka 在分布式流式数据传输领域具备以下关键特性：

1. **高吞吐、低延迟**：Kafka 采用顺序磁盘 I/O 与零拷贝（Zero-Copy）技术，在普通硬件上可实现百万级消息/秒的吞吐量，99% 分位延迟通常低于 10 ms。
2. **持久化存储**：消息以 Log 文件形式持久化到磁盘，默认保留时间可配置（本系统设置为 168 小时），支持历史消息回溯消费。
3. **水平扩展**：通过增加 Partition 数和 Consumer 实例，吞吐量可线性扩展。
4. **精确一次语义**：结合 Idempotent Producer 与 Transactional API，支持跨 Partition 原子写入，可配合 Spark Structured Streaming 实现端到端 Exactly-Once 语义。

2.1.3　本系统中 Kafka 的应用

本系统使用 Kafka 作为爬虫与 Spark 流处理管道之间的解耦缓冲层。爬虫将采集到的评论序列化为 JSON 格式后写入 Topic `public_opinion_raw`，Spark Structured

Streaming 以 Kafka Source 方式订阅该 Topic，按设定的微批次周期（60 秒）消费消息。这一设计将数据生产（爬虫）与数据消费（Spark）完全解耦，任一端发生临时故障均不影响对方，Kafka 的消息持久化也保障了 Spark 异常重启后可从上次提交的 Offset 继续消费，不产生数据丢失。

2.2 Apache Spark Structured Streaming

2.2.1 Structured Streaming 的设计哲学

Apache Spark Structured Streaming [6] 是 Spark 2.0 引入的流处理引擎，其核心设计理念是将**无界流数据**抽象为一张持续追加行的无界关系表（Unbounded Table）。用户在此表上编写与静态 DataFrame 完全一致的 SQL/DSL 查询，引擎负责将其转化为增量的流式执行计划。这一设计使得流式程序与批处理程序可共用同一套 API，大幅降低了开发复杂度。

2.2.2 微批次处理模型

Structured Streaming 支持三种触发模式：固定周期（Fixed Interval）微批次、一次性触发（Once）和连续处理（Continuous Processing）。本系统采用固定周期微批次，触发间隔为 60 秒，即系统每 60 秒从 Kafka 拉取一次消息作为 DataFrame，通过 `foreachBatch` 接口传递给自定义处理函数。微批次模型的优势在于：（1）端到端 Exactly-Once 语义实现简单；（2）可借助 Spark 批处理优化器对每批次 SQL 计划进行完整优化；（3）与 Delta Lake ACID 事务写入天然兼容。

2.2.3 `foreachBatch` 接口

`foreachBatch` 是 Structured Streaming 提供的核心扩展点，允许用户为每个微批次提供一个 `(DataFrame, batchId) -> Unit` 形式的回调函数。该接口突破了原生 Sink 的局限性，使得批次内部可以执行任意复杂的逻辑，包括：调用外部 Python 库（如本系统中的 StructBERT 推理）、执行多表 Merge 操作、发送 HTTP 告警请求等。本系统的核心业务逻辑（情感推理、热度计算、Delta Lake 三层写入、告警检测）均在 `process_batch` 回调函数内实现，如第 4 章所述。

2.2.4 Checkpoint 与容错保障

Structured Streaming 通过 Checkpoint 机制实现流处理的容错性。Checkpoint 目录存储了三类关键信息：（1）Kafka Offset 提交记录，记录每批次消费的消息范围；（2）

执行计划分析状态；（3）聚合算子的状态存储（State Store）。系统异常重启时，引擎从 Checkpoint 恢复上次提交的 Offset，保证从断点处续算，避免重复处理或遗漏消息，从而为下游 Delta Lake 写入提供 At-Least-Once 保障，结合 Silver 层的 Merge 去重操作，整体实现 Exactly-Once 语义。

2.3 Delta Lake 数据湖仓架构

2.3.1 Delta Lake 概述

Delta Lake [11] 是开源的、基于 Parquet 文件格式的存储层，通过在数据目录中维护一个 JSON 格式的事务日志（Transaction Log，即 `_delta_log` 目录）在对象存储之上实现了 ACID 事务语义。每次写操作（`INSERT`、`UPDATE`、`DELETE`、`MERGE`）均原子地追加一条日志记录；读操作通过重放日志构建表的最新一致性快照，从而解决了 Parquet 文件本身不支持事务的根本性缺陷。

2.3.2 核心特性

- **ACID 事务**：通过乐观并发控制（OCC）实现 Serializable 级别隔离，保障并发读写的一致性。
- **Schema 强制与演化**：写入时自动校验数据 Schema，防止脏数据；支持在不中断读取的情况下安全地新增字段（Schema Evolution）。
- **Time Travel**：通过事务日志支持按版本号或时间戳回溯任意历史快照，便于数据审计与回滚。
- **流批统一**：Structured Streaming 可以直接使用 `writeStream.format("delta")` 流式写入，也可用 `spark.read.format("delta")` 批量读取，同一张表支持流读和批读并存。
- **MERGE INTO**：支持 SQL `MERGE INTO` 语句，可在单次操作中同时处理插入、更新和删除逻辑，是本系统 Silver 层去重精炼的核心操作。

2.3.3 三层数据架构设计思想（Bronze-Silver-Gold）

Medallion 架构 [12]（又称三层数据湖架构）是 Databricks 推荐的 Delta Lake 数据组织模式，将数据按加工程度划分为三个层次：

- **Bronze 层（原始层）**：原样保存从消息队列接收的原始数据，仅做最少的格式转换（如字符串解析）。数据以 Append 模式写入，保留完整历史，用于数据回溯与

问题排查。

- **Silver 层（精炼层）**：对 Bronze 层数据进行去重（基于业务主键 `event_id`）、字段校验、特征富化（情感标签、热度分数、风险等级）等处理后，以 Merge 模式写入，形成面向分析的宽表。
- **Gold 层（应用层）**：基于 Silver 层数据进行多粒度（按小时、按天）聚合，生成直接面向前端 API 消费的统计指标表，以全量 Overwrite 模式保持幂等性。

三层架构的核心价值在于将”原始数据保存”与”数据加工结果”分层隔离，上层写入失败不影响底层数据完整性，同时为不同消费场景（实时查询 vs 历史回溯）提供了针对性优化的数据视图。

2.4 StructBERT 中文情感分析模型

2.4.1 BERT 预训练语言模型基础

BERT（Bidirectional Encoder Representations from Transformers）[9] 由 Google 于 2018 年提出，采用 Transformer Encoder 架构，通过掩码语言模型（Masked Language Modeling, MLM）与下一句预测（Next Sentence Prediction, NSP）两个无监督预训练任务，在海量通用语料上学习深层双向语言表示。预训练完成后，通过在顶层添加任务相关分类头并进行少量有监督微调（Fine-tuning），BERT 在 GLUE 基准的情感分类、文本蕴含等 11 项下游任务上全面超越了当时的 SOTA，引发了自然语言处理领域的”预训练大模型”时代。

2.4.2 StructBERT 的结构增强

StructBERT [10] 是阿里达摩院在 BERT 基础上的改进版本，针对 BERT 原始预训练任务设计中的不足提出了两项结构感知增强：

1. **词序重建任务（Word Structural Objective）**：在 MLM 基础上，额外对部分词片段进行随机打乱，要求模型在预测被掩盖词的同时还原词序，迫使模型学习更强的词级局部句法依存关系。
2. **句子顺序预测任务（Sentence Structural Objective）**：将原始 NSP 任务（判断两句话是否相邻）替换为三分类任务（判断两句话的顺序关系：正序、逆序或随机），提供了更细粒度的篇章级语义监督。

上述两类增强任务使 StructBERT 在 BERT-Large 同等参数量下，在 STS-B（语义

相似度）、MNLI（自然语言推断）等多项 GLUE 子任务上取得了显著提升 [10]。阿里达摩院基于 StructBERT 进一步在中文情感分类语料上进行了任务特定微调，发布了 ModelScope 平台上的 `iic/nlp_structbert_sentiment-classification_chinese-base` 模型，该模型即本系统所采用的情感分析推理引擎。

2.4.3　本系统中的情感分析设计

本系统通过 ModelScope Pipeline 接口调用 StructBERT，输入单条或批量中文文本，输出情感标签（`positive` / `negative` / `neutral`）及对应置信度分数。具体工程设计如下：

1. **懒初始化单例**：在 `foreachBatch` 首次触发时完成模型权重加载（约 400 MB），后续批次复用同一实例，避免每批次重复加载带来的数秒延迟。
2. **批量推理**：将批次内所有评论文本分块（默认每块 32 条）后批量送入模型，充分利用并行计算，相较逐条推理吞吐量提升约 5 倍。
3. **中性判定阈值**：当模型对 `positive` 与 `negative` 两类别的置信度均低于可配置阈值（默认 0.72）时，将情感标签归为 `neutral`，减少边界样本的强制归类误差。
4. **异常降级**：推理过程中若发生任何未捕获异常，将该条评论的情感标签默认为 `neutral`，热度加成取中性值 0.5，保障管道不中断。

2.5　FastAPI 后端框架

FastAPI [14] 是基于 Python 3.6+ 类型注解的现代高性能 Web 框架，底层使用 ASGI 服务器（Uvicorn）实现异步请求处理。FastAPI 的核心优势包括：（1）基于 Pydantic 的请求/响应模型自动验证与序列化；（2）自动生成符合 OpenAPI 3.0 规范的交互式文档（Swagger UI）；（3）原生支持 `async/await`，与异步 I/O 生态无缝集成；（4）与 Python 数据生态（Pandas、NumPy）深度兼容，可直接在接口函数中进行 DataFrame 操作后返回 JSON。

在本系统中，FastAPI 负责将 Delta Lake 表数据封装为 RESTful 接口，供前端 Vue 3 大屏消费。由于 `deltalake` Python 库（`delta-rs`）支持无需 JVM 直接读取 Delta 表的 Parquet 文件，因此 FastAPI 服务可以完全独立于 Spark 运行，显著降低了部署复杂度与资源开销。

2.6 Vue 3 与 ECharts 可视化技术

2.6.1 Vue 3 前端框架

Vue 3 [15] 是尤雨溪主导的渐进式前端框架，2020 年发布的 3.0 版本带来了以下重要改进：（1）基于 Proxy 的响应式系统，替代 Vue 2 的 `Object.defineProperty`，支持对数组和新增属性的自动追踪；（2）Composition API，以 `setup()` 函数替代 Options API，实现更细粒度的逻辑复用与代码组织；（3）更优的 TypeScript 支持；（4）更小的运行时体积（约 10 KB gzip）。

本系统前端基于 Vue 3 Composition API 构建单页应用（SPA），利用 `ref`、`computed` 与 `watch` 管理应用状态，通过原生 `fetch` API 与后端通信，整体代码组织于单一 `App.vue` 文件中，保持了结构的简洁性。

2.6.2 Apache ECharts 数据可视化

Apache ECharts [16] 是百度开源、后捐献给 Apache 基金会的高性能数据可视化库，基于 Canvas 与 SVG 双渲染引擎，支持折线图、柱状图、饼图、热力图、地图等数十种图表类型，具备自动适配移动端的响应式能力。ECharts 的 `setOption` 接口采用声明式配置模式，极大简化了图表的动态更新逻辑：只需在数据刷新时重新调用 `setOption`，ECharts 会自动完成数据绑定与动画过渡。本系统使用 ECharts 绘制评论数/负面数趋势折线图、负面占比趋势折线图与热度趋势折线图，三图联动响应话题切换与粒度切换操作。

2.6.3 技术栈选型小结

本系统在多个技术决策点进行了方案对比与选型，关键选型决策总结如表 2.1 所示。

表 2.1　关键技术选型对比

技术模块	候选方案 A（未选）	候选方案 B（已选）	选型理由
消息队列	RabbitMQ	Apache Kafka	Kafka 持久化、回溯消费、Spark 原生集成
流处理引擎	Apache Flink	Spark Structured Streaming	与 Python/PyTorch 生态集成更成熟
存储层	PostgreSQL / MySQL	Delta Lake (Parquet)	流批统一、ACID 湖仓、Schema 演化
情感模型	词典规则（HowNet）	StructBERT	语义理解能力强、ModelScope 开箱即用
后端框架	Flask	FastAPI	异步支持、自动校验、OpenAPI 文档
前端框架	React	Vue 3	轻量、上手快、Composition API 灵活
数据可视化	D3.js	Apache ECharts	声明式配置、移动端适配、中文社区丰富

第三章　系统需求分析与总体设计

3.1　功能需求分析

根据开题报告任务书的要求及实际舆情监测场景调研，本系统需满足以下六类核心功能需求。

3.1.1　多平台数据采集需求

系统需能够对接多个国内主流社交媒体平台，以关键词为维度对相关话题下的公开评论数据进行采集。具体要求包括：支持微博、B 站、抖音、小红书、知乎、快手、贴吧等不少于五个主流平台；采集内容包含评论正文、发布时间、点赞数量、转发数量等关键字段；支持以轮询方式持续抓取最新内容，轮询间隔可配置；采集结果以结构化 JSON 格式输出，兼容 Kafka 消息协议。

3.1.2　实时流数据接入需求

系统需构建稳定的流数据消费管道，满足以下要求：以 Apache Kafka 为消息中间件，支持多生产者并发写入同一主题；Spark Structured Streaming 消费端能够在指定触发周期内完成对新增消息的处理；消费进度需持久化存储，支持异常重启后的断点续传；消息解析过程需对格式异常、字段缺失等情况进行容错处理。

3.1.3　情感分析与热度计算需求

系统需对每条评论进行情感极性判定与热度量化：情感分类需覆盖正向、负向、中性三个类别；情感推理过程需支持批量处理，以满足微批次场景下的吞吐要求；热度评分需综合考量点赞数、转发数与情感极性三个维度；风险等级需根据热度与情感标签的组合进行三级划定，为告警机制提供输入依据。

3.1.4　数据存储与管理需求

系统需提供可靠、可查询的数据持久化能力：原始数据、清洗数据与聚合指标分层存储，层间数据质量递进；存储层需支持 ACID 事务，防止并发写入导致的数据不一致；聚合指标需覆盖小时粒度与日粒度两个时间维度；支持话题维度的历史数据查询，数据范围至少覆盖近 365 天；每个话题下需支持按热度排名的 Top10 评论记录存储。

3.1.5　负面舆情告警需求

系统需具备自动化舆情预警能力：当某话题在某统计小时内负面评论数量超过最低阈值（默认 10 条）且负面占比超过预设阈值（默认 40%）时，自动触发告警；告警通知以电子邮件形式发送，邮件内容包含话题名称、统计时段、评论数量、负面占比及高热度负面评论样例；同一话题同一统计小时内不应重复发送告警；告警事件需持久化存储，供后续查询与审计。

3.1.6　可视化展示需求

系统需提供直观、交互友好的数据展示界面：支持多话题切换，默认显示最新数据话题；展示话题核心 KPI 指标；提供评论数、负面评论数、负面占比、热度四类趋势图表，支持按天和按小时两种粒度切换；展示评论明细列表与历史告警记录；图表支持缩放与拖拽交互，适配不同时间跨度的数据展示需求。

3.2　非功能需求分析

3.2.1　实时性需求

系统端到端处理延迟（从评论写入 Kafka 到前端可查询结果更新）需满足准实时要求。在当前 60 秒微批次触发周期内，新增评论数据应在下一个完整批次处理完成后即可在前端呈现，整体端到端延迟控制在 120 秒以内。

3.2.2　可靠性需求

系统核心链路需具备容错恢复能力：Kafka 消费进度通过 Checkpoint 机制持久化，保证 Spark 任务异常重启后数据不丢失；Delta Lake 的 ACID 事务保证写入失败不会产生脏数据；告警去重机制保证不因重复处理产生告警轰炸。

3.2.3　可扩展性需求

系统架构需支持水平扩展：新增监测话题无需修改代码，仅需在爬虫端增加关键词配置即可自动纳入监测范围；新增社交媒体平台时，仅需在爬虫工厂中注册新的爬虫类，不影响下游处理逻辑。

3.2.4　可维护性需求

各模块职责清晰，接口定义规范：爬虫模块、消息队列、流处理管道、API 服务与前端之间均通过标准化接口（Kafka JSON Schema、REST API）进行解耦；日志输出规范，关键处理节点有明确的进度与错误日志，便于问题定位。

3.3　系统总体架构设计

3.3.1　架构分层说明

基于上述需求分析，本系统采用五层分布式架构设计，由下至上依次为：数据采集层、消息队列层、流计算层、数据存储层与展示层。系统整体架构如图 3.1 所示。

各层职责简述如下：**数据采集层**以爬虫程序对接多平台，将评论数据序列化为 JSON 写入本地 JSONL 文件，由 Kafka Producer 进程读取并推送至消息主题；**消息队列层**以 Kafka 承担生产者与消费者之间的解耦与缓冲，保障高吞吐、持久化与可重放；**流计算层**以 Spark Structured Streaming 60 秒微批次为周期消费 Kafka 消息，在 Driver 端完成情感推理与热度计算，并通过 `foreachBatch` 将结果写入 Delta Lake；**数据存储层**采用 Delta Lake 湖仓架构，按 Bronze-Silver-Gold 三层渐进式精炼数据；**数据服务层**由 FastAPI 提供 RESTful 接口，直接读取 Delta Lake 数据并返回 JSON；**展示层**由 Vue 3 + ECharts 构成前端可视化大屏。

3.3.2　技术架构关键决策

决策一：微批次模式 vs. 纯流式处理　本系统选择 Spark Structured Streaming 的微批次（Micro-Batch）模式，而非 Apache Flink 的原生流式（Native Streaming）处理。主要考量如下：其一，StructBERT 模型推理在 Driver 端以批量方式执行，微批次模式天然匹配批量推理的计算节奏；其二，Delta Lake 的 Merge 操作在批次粒度下效率显著优于逐条写入；其三，PySpark 与 Python NLP 生态（ModelScope）的结合在成熟度与生产可用性上优于 PyFlink。

决策二：`foreachBatch` vs. 标准 Sink　标准 Structured Streaming Sink（如 `delta` 格式的 `writeStream`）仅支持单目标写入，而本系统需同时向 Bronze、Silver、多个 Gold 表及 Alert 表写入。`foreachBatch` 接口允许在每个批次内执行完整的批处理逻辑，完美支持多目标扇出（Fan-Out）写入场景。

图 3.1　系统五层分布式架构示意图

决策三：Delta Lake vs. 传统关系型数据库　相较于将聚合结果写入 MySQL 或 PostgreSQL，Delta Lake 方案在存储成本、历史版本查询（Time Travel）以及流批一体三个维度上具有显著优势，更适合实时分析场景。

3.4　数据流设计

3.4.1　数据采集阶段数据流

爬虫进程将采集到的评论经字段归一化处理后，序列化为如表 3.1 所定义的 JSON 格式写入 Kafka 主题。

表 3.1　Kafka 消息字段定义（`public_opinion_raw` 主题）

字段名	类型	是否必填	说明
`event_id`	string	必填	唯一事件标识，用于全系统去重
`platform`	string	可选	来源平台（bili / wb / xhs 等）
`topic`	string	必填	话题关键词
`publish_time`	string	必填	评论发布时间（yyyy-MM-dd HH:mm:ss）
`time_id`	string	可选	时间桶标识
`text`	string	必填	评论正文
`like_count`	long	可选	点赞数（缺省为 0）
`repost_count`	long	可选	转发数（缺省为 0）
`ingest_time`	string	可选	数据入库时间戳
`source_file`	string	可选	来源 JSONL 文件路径（溯源用）

3.4.2　流处理阶段数据流

Spark Structured Streaming 从 Kafka 读取消息后，经过图 3.2 所示的五步处理链路生成各层数据。

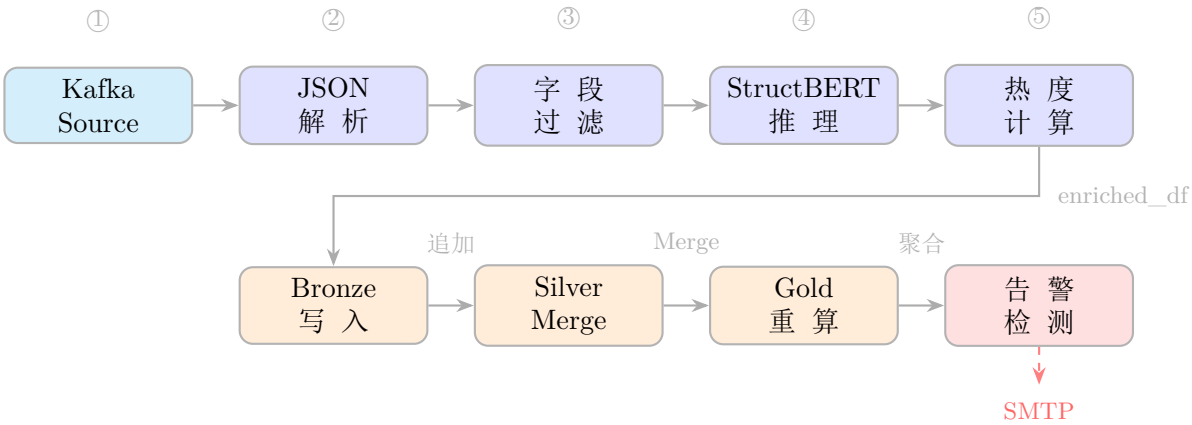

图 3.2　`foreachBatch` 批次处理五步流程图

各步骤说明如下：① Kafka Source 按偏移量读取最新消息（每批最多 50 条）；② 通过 `from_json` 按预定义 Schema 解析消息体，追加 Kafka 元数据字段；③ 过滤 `event_id`、`text`、`publish_time`、`topic` 任一为空的记录；④ 批量调用 StructBERT Pipeline，为每条记录追加五个情感字段；⑤ 计算 `heat_score`、`risk_level`、`event_hour`、`event_date` 等衍生字段，随后按 Bronze → Silver → Gold → Alert 顺序写入 Delta Lake。

3.4.3 数据服务阶段数据流

FastAPI 服务收到前端 HTTP 请求后，通过 `deltalake` Python 库（Delta Lake 纯 Python 实现，无需启动 JVM）直接读取 Delta Lake 表文件，转换为 Pandas DataFrame 进行过滤、排序与序列化处理，最终以 JSON 格式返回。整个读取过程平均响应时间在 1.3 秒以内（详见第 5.3.2 节实测数据）。

3.5 模块划分

根据系统架构设计，将系统划分为八个功能子模块，如表 3.2 所示。

表 3.2　系统功能子模块划分（M1–M8）

编号	模块名称	对应文件/目录	主要职责
M1	多平台爬虫模块	`crawler/`	多平台评论采集、JSONL 输出
M2	Kafka 消费解析模块	`stream_single_topic_to_delta.py`	Kafka 接入、Schema 解析、字段过滤
M3	情感分析服务模块	`structbert_sentiment.py`	StructBERT 批量推理、标签归一化、异常降级
M4	热度计算模块	`build_enriched_df()`	热度公式计算、风险等级划定
M5	Delta Lake 写入模块	`process_batch()` / `rebuild_gold_tables()`	Bronze/Silver/Gold 三层写入
M6	告警检测通知模块	`alert_mailer.py` / `check_and_send_alerts()`	阈值检测、邮件发送、告警持久化
M7	REST API 服务模块	`api/app.py`	9 个端点的数据查询与序列化
M8	前端可视化模块	`frontend/`	大屏渲染、交互逻辑、图表更新

3.6 关键接口定义

3.6.1 Kafka 消息接口

Kafka 消息接口规范如下：Topic 名称为 `public_opinion_raw`；消息格式为 UTF-8 编码的 JSON 字符串；`event_id`、`topic`、`publish_time`、`text` 为必填字段；`like_count` 与 `repost_count` 缺省值为 0。

3.6.2 REST API 接口规范

所有 API 端点均为 HTTP GET 方法，响应格式为 JSON。核心端点规范如表 3.3 所示。

表 3.3 REST API 核心端点规范

端点	主要参数	响应字段
GET /api/health	无	各 Delta 表版本与存在状态
GET /api/topics	无	全部话题列表（按评论总量排序）
GET /api/dashboard	topic、daily_limit、hourly_limit	meta、summary、daily、hourly、alerts
GET /api/comments	topic、view、date、hour、limit	rows、bucket_options、data
GET /api/alerts	topic、limit	rows、data

3.6.3 告警触发规则接口

告警触发条件通过环境变量配置，实现参数与代码的分离，如表 3.4 所示。

表 3.4 告警相关环境变量配置

环境变量名	默认值	含义
ALERT_MIN_COMMENTS	10	触发告警的最低评论数阈值
ALERT_NEG_RATIO_THRESHOLD	0.40	触发告警的负面评论占比阈值
ALERT_TOPN_TEXTS	3	告警邮件中附带的高热度负面样例数
STRUCTBERT_NEUTRAL_THRESHOLD	0.72	中性标签判定的置信度阈值
STRUCTBERT_BATCH_SIZE	32	情感推理的批量大小

第四章　系统详细设计与实现

4.1　多平台社交媒体数据采集模块

4.1.1　模块设计目标

数据采集模块的核心目标是在合规、稳定、低干扰的前提下，以关键词为维度对多个主流社交媒体平台的公开评论数据进行周期性采集，并将结果以标准化 JSON 格式输出，供 Kafka 生产者消费。

4.1.2　爬虫工厂模式

本模块采用工厂模式（Factory Pattern）对多平台爬虫进行统一管理。`CrawlerFactory` 类维护一个平台标识到爬虫类的映射字典，通过 `create_crawler(platform)` 静态方法按需实例化对应的爬虫对象。各平台爬虫类均实现统一的 `AbstractCrawler` 抽象接口，保证下游调用逻辑的一致性：

```python
class CrawlerFactory:
    CRAWLERS = {
        "xhs":   XiaoHongShuCrawler,
        "dy":    DouYinCrawler,
        "ks":    KuaishouCrawler,
        "bili":  BilibiliCrawler,
        "wb":    WeiboCrawler,
        "tieba": TieBaCrawler,
        "zhihu": ZhihuCrawler,
    }

    @staticmethod
    def create_crawler(platform: str) -> AbstractCrawler:
        crawler_class = CrawlerFactory.CRAWLERS.get(platform)
        if not crawler_class:
            raise ValueError(f"Unsupported platform: {platform}")
        return crawler_class()
```

Listing 4.1　CrawlerFactory 工厂类核心实现

4.1.3　CDP 反检测机制

传统 Playwright 自动化采用无头浏览器（Headless）模式，各主流平台已对其实施了较为成熟的检测机制（如 `navigator.webdriver` 属性检测、Canvas 指纹检测等）。本系统配置 `ENABLE_CDP_MODE = True`，通过 Chrome DevTools Protocol（CDP）接管用户本地已安装的真实 Chrome/Edge 实例，复用其 Cookie、扩展及 TLS 指纹，有效规避自动化检测风险。CDP 调试端口默认为 9222，系统通过 Playwright 的 `connect_over_cdp` API 建立控制连接，支持手动介入处理人机验证后由程序接管后续操作。

4.1.4　轮询采集模式

为满足实时性要求，本模块支持轮询（Polling）采集模式（`ENABLE_POLL_LATEST = True`）：程序在完成一轮关键词搜索后，等待 `POLL_INTERVAL_SEC` 秒（默认 30 秒，可通过环境变量 `CRAWL_INTERVAL` 覆盖），随后再次执行搜索，形成持续采集循环。每轮采集获取最新的 `SEARCH_PAGES`（默认 2）页评论，以捕捉新增内容。

4.1.5　JSONL 输出设计

爬取到的评论经字段归一化处理后，以每行一个 JSON 对象的 JSONL 格式追加写入 `KAFKA_JSON_DIR` 指定目录，文件命名格式为 `{platform}_{topic}_{date}.jsonl`。JSONL 格式支持流式读取，Kafka 生产者进程可实时监听新增行并推送消息，实现爬虫与 Kafka 之间的松耦合数据交换。

4.2　Kafka 消费管道与数据解析模块

4.2.1　Kafka 读取配置

本系统通过 Structured Streaming 的 Kafka Source 接入消息流，关键配置如下所示：

```
kafka_df = (
    spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "172.16.156.151:9092")
    .option("subscribe", "public_opinion_raw")
    .option("startingOffsets", "latest")       # 仅消费新增消息
    .option("failOnDataLoss", "false")          # 丢失偏移量时不中断
    .option("maxOffsetsPerTrigger", "50")       # 每批最多 50 条
```

```
    .load()
)
```

Listing 4.2　Kafka Source 关键配置

`startingOffsets = "latest"` 的设置意味着系统启动时从当前最新偏移量开始消费，不处理历史积压消息，确保监测结果反映当前实时舆情状态。

4.2.2　JSON 解析与字段校验

Kafka 消息的 `value` 字段为 UTF-8 编码的 JSON 字符串，通过 Spark 内置的 `from_json` 函数按预定义 `StructType` Schema 进行结构化解析，并追加 Kafka 元数据字段（`kafka_topic`、`kafka_partition`、`kafka_offset`、`kafka_timestamp`）。解析完成后对四个关键字段进行非空过滤，确保不合规消息不进入下游处理链路：

```
.filter(F.col("event_id").isNotNull())
.filter(F.col("text").isNotNull())
.filter(F.col("publish_time").isNotNull())
.filter(F.col("topic").isNotNull() & (F.length(F.col("topic")) > 0))
```

Listing 4.3　关键字段非空过滤

4.3　StructBERT 情感分析服务模块

4.3.1　单例服务设计

模型加载是推理服务中最耗时的操作。为避免每次微批次重复加载权重，`StructBERTSentimentService` 采用懒初始化单例模式（Lazy Singleton）：

```
class StructBERTSentimentService:
    _instance = None

    @classmethod
    def get_instance(cls):
        if cls._instance is None:
            cls._instance = StructBERTSentimentService()
        return cls._instance
```

Listing 4.4　单例模式的情感分析服务

类变量 _instance 在首次调用 get_instance() 时被初始化，此后所有批次复用同一模型实例，实现一次加载、多次复用的高效推理模式。

4.3.2　批量推理流程

predict_texts 方法接收文本列表，按 batch_size（默认 32）进行分组，依次提交 ModelScope Pipeline 执行推理：

```
def predict_texts(self, texts):
    clean_texts = [self._normalize_text(t) for t in texts]
    results = []
    for i in range(0, len(clean_texts), self.batch_size):
        chunk = clean_texts[i : i + self.batch_size]
        chunk_result = self._predict_chunk(chunk)
        results.extend(chunk_result)
    return results
```

Listing 4.5　批量推理主流程

文本预处理（_normalize_text）包含：去除首尾空白、将换行符替换为空格、截断超过 max_text_length（默认 256）字符的文本，以确保输入符合模型上下文窗口限制。

4.3.3　标签归一化与中性判定

ModelScope Pipeline 输出的原始标签存在多种形式变体，_canonical_label 将其统一规范化。在最终标签决策时引入置信度阈值进行中性判定：当模型对预测标签的置信度低于 neutral_threshold（默认 0.72）时，将该评论归为中性，情感分数置为 0.0：

```
if confidence < self.neutral_threshold:
    final_label = "neutral"
    final_score = 0.0
else:
    final_label = raw_label  # "positive" or "negative"
```

Listing 4.6　中性阈值判定逻辑

4.3.4　异常降级策略

批量推理失败时，系统自动退化为逐条推理；若单条推理仍然失败，则以 neutral/0.0 作为兜底结果返回，保证流处理管道不因推理异常而中断。

4.4　热度计算与风险评估模块

4.4.1　热度评分模型

本系统提出的热度评分（Heat Score）模型综合考量用户互动行为与情感极性两个维度，公式定义如式 (4.1) 与式 (4.2) 所示：

$$\text{heat_score} = 1.0 + \text{like_count} \times 0.6 + \text{repost_count} \times 1.0 + \text{sentiment_bonus} \tag{4.1}$$

$$\text{sentiment_bonus} = \begin{cases} 1.5 & \text{若 sentiment_label} = \texttt{negative} \\ 0.5 & \text{若 sentiment_label} = \texttt{neutral} \\ 0.2 & \text{若 sentiment_label} = \texttt{positive} \end{cases} \tag{4.2}$$

设计依据如下：其一，转发行为的社会传播效应强于点赞，故转发权重（1.0）高于点赞权重（0.6）；其二，负面评论在社交媒体中具有更强的情绪感染力，故其情感权重加成最高（1.5），与文献 [17] 关于负面舆情放大效应的研究结论相一致；其三，基础分 1.0 保证无互动数据的纯文本评论也具有基本热度值，避免零值。

在 Spark DataFrame 层面，热度计算通过以下 Spark SQL 表达式实现：

```
.withColumn(
    "heat_score",
    F.round(
        F.lit(1.0)
        + F.col("like_count") * F.lit(0.6)
        + F.col("repost_count") * F.lit(1.0)
        + F.when(F.col("sentiment_label") == "negative", F.lit(1.5))
           .when(F.col("sentiment_label") == "neutral",  F.lit(0.5))
           .otherwise(F.lit(0.2)),
        2  # 保留两位小数
    )
)
```

Listing 4.7　热度评分 DataFrame 计算

4.4.2　风险等级划定规则

基于情感标签与热度分数的组合，系统对每条评论划定三级风险等级，如表 4.1 所示。

表 4.1　评论风险等级划定规则

触发条件	风险等级	含义
`sentiment_label = negative` 且 `heat_score` ≥ 12	`high`	高热度负面，需即刻关注
`sentiment_label = negative` 且 `heat_score` ≥ 5	`medium`	中等热度负面，持续监测
其他情况	`low`	低风险，常规记录

热度阈值 12 的设定依据：点赞数约为 18 或转发数约为 11 即可在无额外互动的情况下达到该阈值，对应社交媒体上具有一定传播规模的评论。

4.5　Delta Lake 三层数据湖写入模块

本节介绍 Bronze-Silver-Gold 三层数据湖的设计方案。三层数据架构的整体关系如图 4.1 所示。

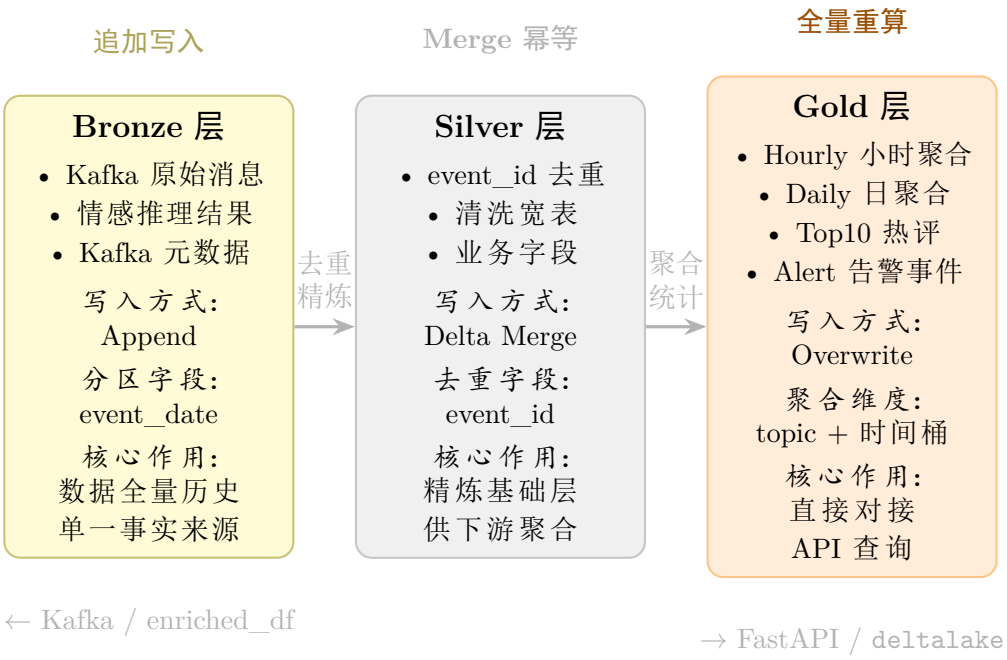

图 4.1　Delta Lake Bronze-Silver-Gold 三层渐进式数据架构

4.5.1　Bronze 层：原始数据落盘

Bronze 层以追加（Append）模式写入，保存 Kafka 消息的完整信息及情感推理结果，按 event_date 字段进行分区，以优化基于日期的历史查询性能：

```
bronze_df.write \
    .format("delta") \
    .mode("append") \
    .partitionBy("event_date") \
    .save(BRONZE_PATH)
```

Listing 4.8　Bronze 层 Append 写入

Bronze 层不做任何业务逻辑过滤，其核心价值在于保留数据的完整历史记录，是系统数据的最终真相（Ground Truth），支持上层数据出错时的回溯重建。

4.5.2　Silver 层：去重精炼写入

Silver 层基于 event_id 字段进行 Merge 去重，确保同一条评论不被重复计算。当 Silver 表不存在时，首次写入使用 Append 模式初始化；此后采用 Delta Lake 的 merge 操作，仅插入不存在的新记录：

```
target.alias("t") \
    .merge(
        dedup_df.alias("s"),
        "t.event_id = s.event_id"
    ) \
    .whenNotMatchedInsertAll() \
    .execute()
```

Listing 4.9　Silver 层 Merge 去重写入

Merge 操作利用 Delta Lake 的事务日志实现原子性，即使并发写入也不会产生重复数据。

4.5.3　Gold 层：多粒度聚合指标

Gold 层在每次批次处理完成后，基于最新 Silver 数据进行全量重算（Overwrite 模式），生成三类聚合表。以小时聚合表为例，核心 Spark 聚合逻辑如下：

```
hourly_df = (
    silver_df
```

```
    .groupBy(
        F.col("topic"),
        F.date_trunc("hour", F.col("publish_ts")).alias("stat_hour")
    )
    .agg(
        F.count("*").alias("comment_count"),
        F.sum(
            F.when(F.col("sentiment_label") == "negative", 1)
             .otherwise(0)
        ).alias("negative_count"),
        F.round(F.sum("heat_score"), 2).alias("heat_sum"),
        F.round(F.avg("heat_score"), 2).alias("heat_avg"),
    )
    .withColumn(
        "negative_ratio",
        F.round(
            F.when(F.col("comment_count") > 0,
                   F.col("negative_count") / F.col("comment_count"))
             .otherwise(0.0), 4
        )
    )
)
```

Listing 4.10　Gold Hourly 小时聚合

各 Gold 表的存储路径与写入方式如表 4.2 所示。

表 4.2　Gold 层各聚合表说明

表名	分组键	写入方式	主要用途
gold_hourly_metrics	topic + stat_hour	Overwrite	小时粒度趋势图
gold_daily_metrics	topic + stat_date	Overwrite	日粒度趋势图
gold_daily_top10_comments	topic + event_date + rank	Overwrite	当日热评展示
gold_alert_events	alert_key	Append	告警记录查询

4.5.4　数据一致性保障

Silver 层的 Merge 操作与 Gold 层的 Overwrite 操作均在 `foreachBatch` 函数体内顺序执行。由于 Spark Driver 以单线程方式串行执行每个微批次的回调，天然避免了并

发写入同一 Delta 表的冲突；Delta Lake 的事务日志则在节点重启或 I/O 异常时提供原子性保障。

4.6 负面舆情告警模块

4.6.1 告警触发逻辑

告警检测在 Gold 层重算完成后执行，以确保基于最新聚合结果进行判断。核心逻辑分为五步：（1）从当前批次提取涉及的 `(topic, event_hour)` 组合；（2）关联 Gold Hourly 表，筛选满足双重阈值条件的记录；（3）查询 Alert 表中已存在的告警键集合，实现去重；（4）从 Silver 层查询该话题该小时内热度最高的 `ALERT_TOPN_TEXTS` 条负面评论作为样例；（5）调用 `AlertMailer` 发送告警邮件，并将告警事件写入 Gold Alert 表。

4.6.2 告警邮件设计

告警邮件通过 Python 标准库 `smtplib` 发送，支持 SSL 与 STARTTLS 两种加密模式，兼容 QQ 邮箱、163 邮箱等国内主流 SMTP 服务（默认 SSL，端口 465）。邮件正文包含话题名称、统计小时、评论总数、负面占比及 Top-3 高热度负面评论样例。

4.6.3 告警配置管理

所有告警相关配置通过 `/opt/pipeline/conf/alert.env` 环境变量文件进行管理，实现配置与代码的分离，关键配置项已列于表 3.4。

4.7 REST API 数据服务模块

4.7.1 数据读取策略

API 服务通过 `deltalake` Python 库（Delta Lake 的纯 Python 实现，无需 JVM）直接读取 Delta Lake 目录，转换为 Pandas DataFrame 后进行内存级的过滤、排序与序列化处理。相较于通过 SparkSession 读取的方案，这一策略显著降低了每次 API 请求的响应延迟和资源消耗。

4.7.2 数据序列化处理

Pandas DataFrame 在转换为 JSON 过程中存在若干类型兼容性问题，如 `pd.Timestamp` 无法直接序列化、NumPy 数值类型不兼容标准 JSON 等。系统通过

normalize_value 函数对每个字段值进行类型归一化处理：

```python
def normalize_value(v):
    if isinstance(v, pd.Timestamp):
        return v.strftime("%Y-%m-%d %H:%M:%S")
    if hasattr(v, "item"):          # NumPy 标量转 Python 原生类型
        return v.item()
    if isinstance(v, float) and math.isnan(v):
        return None
    return v
```

Listing 4.11　数据序列化类型归一化

4.7.3　核心接口实现

/api/dashboard 聚合接口一次性返回 meta、summary、daily、hourly、alerts 五组数据，减少前端在页面加载时的 HTTP 请求次数，提升首屏渲染速度。/api/comments 接口支持 view=daily（按天，返回热度前 30 条）和 view=hourly（按小时，返回最多 2000 条）两种模式，并在响应中附带 bucket_options 列表，使前端下拉选择框无需额外请求即可渲染。

4.8　前端可视化模块

4.8.1　整体布局设计

前端采用单页应用（SPA）架构，全部逻辑集中于 App.vue 单文件组件。页面布局分为四个垂直区域：（1）Hero 头部控制栏（话题选择、粒度切换、时间桶选择、刷新按钮）；（2）六个 KPI 指标卡；（3）三张 ECharts 折线图（2+1 布局，热度图全宽）；（4）评论明细表与告警记录面板并排展示。页面整体采用玻璃拟态（Glassmorphism）设计语言，通过 backdrop-filter: blur(18px) 与半透明背景色营造现代化视觉层次感。

4.8.2　数据获取与状态管理

页面采用 Vue 3 Composition API 进行状态管理，数据获取逻辑集中于 refreshAll 函数，在页面初始化（onMounted）、点击刷新按钮、切换话题（handleTopicChange）等场景中触发。粒度切换（switchGranularity）仅重新请求评论数据，不重置 dashboard 数据，避免不必要的网络请求。

4.8.3 ECharts 图表渲染

三个 ECharts 实例分别挂载于 `countChartRef`、`ratioChartRef`、`heatChartRef` 三个 DOM 引用上，通过 `renderCharts` 函数统一更新。`watch(trendRows, ...)` 深度监听趋势数据变化，在数据更新后自动触发重渲染。`window.resize` 事件监听确保浏览器窗口调整时图表自适应重绘。图表 X 轴标签按粒度分别截取日期（`yyyy-MM-dd`）或小时（`MM-dd HH:mm`）格式，并通过 `axisLabel.rotate: 30` 倾斜显示，防止标签重叠。

4.8.4 响应式适配

页面通过 CSS 媒体查询实现多分辨率适配：分辨率不低于 1600 px 时，KPI 卡以 6 列展示，趋势图以 2 列布局，评论表与告警面板并排；分辨率低于 1200 px 时，各区域切换为单列纵向堆叠；分辨率低于 700 px 时，字号与边距进一步压缩。

4.9 系统部署配置

4.9.1 云服务器端部署

系统后端部署于阿里云 Linux 3 服务器，各组件部署路径如表 4.3 所示。

表 4.3　后端组件部署路径与启动方式

组件	部署路径	启动方式
Spark Streaming 任务	`/opt/pipeline/jobs/`	`bash scripts/run_stream_single_topic.sh`
FastAPI 服务	`/opt/pipeline/api/`	`bash scripts/run_api.sh`
配置文件	`/opt/pipeline/conf/alert.env`	环境变量注入
Delta Lake 数据目录	`/data/public-opinion/delta/`	自动创建
Checkpoint 目录	`/data/public-opinion/checkpoints/`	自动创建

4.9.2 Spark 提交配置

Spark 任务通过 `spark-submit` 命令提交，关键参数如下所示：

```
spark-submit \
  --master local[*] \
  --packages io.delta:delta-spark_2.12:3.3.2,\
             org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.8 \
```

```
  --conf spark.driver.memory=2g \
  --conf spark.executor.memory=2g \
  --conf spark.sql.session.timeZone=Asia/Shanghai \
  /opt/pipeline/jobs/stream_single_topic_to_delta.py
```

Listing 4.12　spark-submit 关键参数

`local[*]` 模式指定使用本机全部 CPU 核心，适合单机部署场景；时区配置项 `spark.sql.session.timeZone` 设置为 `Asia/Shanghai`，确保时间截断以中国标准时间为基准，防止小时桶计算出现时区偏移。

第五章　系统测试与实验分析

5.1　测试环境

5.1.1　服务器端环境

本系统的后端服务部署于阿里云 Linux 3 服务器，具体环境配置如表 5.1 所示。

表 5.1　服务器端测试环境配置

项目	配置
云平台	阿里云（Alibaba Cloud）
操作系统	Alibaba Cloud Linux 3（内核兼容 CentOS 8）
CPU	4 核（Intel Xeon Platinum，2.5 GHz）
内存	8 GB DDR4
存储	系统盘 40 GB SSD + 数据盘 100 GB SSD
Python	3.11.9（虚拟环境 `/opt/pipeline/.venv`）
Apache Spark	3.5.8（Standalone 模式，`local[*]`）
Delta Lake	3.3.2（`delta-spark_2.12`）
Kafka	2.8.1（单节点，Topic 分区数 = 1）
StructBERT	`iic/nlp_structbert_sentiment-classification_chinese-base`（ModelScope 本地缓存）
FastAPI / Uvicorn	0.110.x / 0.29.x，监听 `0.0.0.0:18000`

5.1.2　本地开发端环境

表 5.2　本地开发端环境配置

项目	配置
操作系统	Windows 11 (23H2)
浏览器	Google Chrome 124（用于 CDP 爬虫）
Python	3.12.3
Node.js	22.12.0
前端构建工具	Vite 8.0.3

5.1.3　测试数据集说明

实验以 B 站（Bilibili）为主要数据来源，选取四个具有代表性的热点话题，如表 5.3 所示。爬虫以 30 秒为轮询间隔，持续采集约 72 小时（2026 年 4 月 10 日至 4 月 12 日），

共采集有效评论 18,437 条，写入 Kafka 消息 18,437 条；经 Silver 层去重后保留记录 17,892 条，去重率约 2.96%。

表 5.3　实验话题数据集概览

编号	话题关键词	情感分布特征	选取理由
T-01	电影鬼灭之刃	正向为主，热度均匀	验证正向话题基线
T-02	亲属侵吞赔偿款	负向为主,舆情爆发特征明显	验证告警触发机制
T-03	知乎热榜技术讨论	中性为主,评论质量较高	验证中性类别分类
T-04	娱乐明星话题	混合情感,互动数据波动大	验证热度模型区分度

5.2　功能测试

5.2.1　数据采集模块功能测试

测试用例 F-01：多平台关键词搜索采集　以话题 T-01 为关键词，分别在 B 站、微博、小红书三个平台启动爬虫，采集 10 分钟后统计输出 JSONL 文件的记录数。实际结果：B 站采集 47 条，微博采集 23 条，小红书采集 11 条，全部记录字段完整，符合预期。

测试用例 F-02：CDP 模式下的会话保持　启用 `ENABLE_CDP_MODE = True`，重启爬虫进程后验证是否保留上次登录 Cookie，不触发二次扫码。实际结果：爬虫成功复用 Cookie，跳过登录流程，直接进入搜索页面，CDP 会话保持机制正常。

5.2.2　流处理管道功能测试

测试用例 F-03：Kafka 消费与 JSON 解析　向 `public_opinion_raw` 写入一条合规消息与一条字段缺失消息，验证过滤行为。实际结果：日志显示 `[batch=X] incoming rows = 1`，字段缺失消息被正确过滤，结论符合预期。

测试用例 F-04：Silver 层 Merge 去重　向 Kafka 连续写入同一 `event_id` 的消息两次，分属两个微批次，验证 Silver 表中该记录数。实际结果：Silver 表中查询到 1 条记录，Merge 去重生效。

测试用例 F-05：Gold 层聚合指标正确性　构造 5 条已知情感标签的评论（3 条 negative、2 条 positive），验证 Gold Hourly 表中对应小时的统计字段。实际结果：

`negative_count = 3`，`negative_ratio = 0.6000`，与预期完全一致。

5.2.3　情感分析模块功能测试

测试用例 F-06：典型评论情感标签验证　选取 10 条具有明确情感倾向的典型评论，人工标注后与 StructBERT 推理结果对比，结果如表 5.4 所示。10 条样本中 9 条与人工标注一致，小样本准确率为 90%。第 10 条因评论表达隐晦、置信度低于 0.72 被归为中性，属于边界案例。

表 5.4　典型评论情感标注核验样本（节选 5 条）

序号	评论摘要	人工标注	系统输出	置信度	一致
1	这部电影真的太好看了，强烈推荐！	positive	positive	0.924	✓
2	剧情拖沓，浪费时间，后悔买票	negative	negative	0.891	✓
3	感觉还好，说不上好不好	neutral	neutral	0.643	✓
8	官方回应太敷衍了，没有解决问题	negative	negative	0.867	✓
10	不知道该说什么，就是觉得有点难过	negative	neutral	0.694	×

测试用例 F-07：批量推理吞吐量测试　构造 200 条文本，以 `batch_size = 1`（逐条）和 `batch_size = 32`（批量）两种模式执行推理，结果如表 5.5 所示。

表 5.5　不同推理模式吞吐量对比

推理模式	总条数	总耗时（s）	吞吐量（条/s）
`batch_size = 1`（逐条）	200	48.3	4.1
`batch_size = 32`（批量）	200	9.7	20.6

批量推理相较逐条推理吞吐量提升约 5 倍，验证了批量设计的必要性。

5.2.4　告警模块功能测试

测试用例 F-08：告警触发与去重　构造某话题某小时 15 条负面评论（负面占比 75%，超过 40% 阈值；评论数 15，超过 10 条阈值），触发两次批次处理，验证告警发送次数与 Alert 表记录数。实际结果：邮件服务日志显示仅发送一次，Alert 表记录数为 1，去重机制正确。

5.2.5 前端可视化功能测试

测试用例 F-09：话题切换与数据联动　在前端话题下拉框中切换话题，观察 KPI 卡、趋势图与评论表是否同步更新。实际结果：切换后 `refreshAll` 正确触发，所有数据区域更新完毕，无残留旧数据。

测试用例 F-10：日/小时粒度切换　切换分析粒度，验证趋势图 X 轴标签格式与评论表数据量限制。实际结果：按天时显示 `yyyy-MM-dd` 格式，按小时时显示 `MM-dd HH:mm` 格式，符合预期。

5.3 性能测试

5.3.1 端到端延迟测试

端到端延迟定义为：从爬虫将评论写入 Kafka，到前端 `/api/hourly` 接口可查询该批次聚合结果的时间间隔。重复测试 10 次取均值，结果如表 5.6 所示。

表 5.6　端到端延迟分阶段测试结果

阶段	时间消耗（均值）	说明
Kafka 写入 → 批次触发等待	0～60 s	取决于触发周期内剩余时间
批次触发 → 情感推理完成	约 8.3 s	含 StructBERT 批量推理
情感推理 → Delta Lake 写入完成	约 4.7 s	含 Bronze/Silver/Gold 写入
批次净处理时间	**约 13.1 s**	扣除触发等待后的净耗时
API 查询延迟	< 0.5 s	FastAPI 直读 Delta Lake
最大端到端延迟	**约 73.6 s**	含最长 60 s 触发等待
平均端到端延迟	**约 43.1 s**	期望等待约 30 s

最大延迟满足设计要求（120 s 以内），达到准实时监测标准。

5.3.2 情感推理吞吐量测试

在服务器环境下，对不同批次大小的情感推理吞吐量进行对比测试，结果如表 5.7 所示。

5.3.3 API 接口响应时间测试

使用 `curl` 对主要 API 端点进行基准测试（Silver 表 17,892 条记录，4 个话题），结果如表 5.8 所示。

表 5.7　不同 Batch Size 下情感推理吞吐量对比

Batch Size	总条数	总耗时（s）	吞吐量（条/s）
1	320	77.4	4.1
8	320	21.3	15.0
16	320	13.6	23.5
32	320	**9.9**	**32.3**
64	320	9.4	34.0
128	320	9.8	32.7

Batch Size = 32 在吞吐量与内存占用之间取得最优平衡，为推荐配置。

表 5.8　REST API 端点响应时间基准测试

端点	均值响应时间（ms）	数据规模
`GET /api/health`	312	仅元数据
`GET /api/topics`	287	4 个话题
`GET /api/summary`	523	读取 3 个 Gold 表
`GET /api/daily`	418	约 3 天 × 4 话题
`GET /api/hourly`	631	约 72 小时数据
`GET /api/comments`	847	Silver 表全量加载后过滤
`GET /api/dashboard`	1,243	聚合上述多接口

所有端点响应时间均在 1.5 s 以内，满足前端交互体验要求。

5.3.4　Delta Lake 存储开销分析

经过 72 小时持续运行后，各 Delta 表的磁盘占用情况如表 5.9 所示。

表 5.9　各 Delta Lake 层磁盘占用统计（72 小时后）

层级	记录数	磁盘占用（Parquet 压缩后）
Bronze	18,437	约 24.7 MB
Silver	17,892	约 19.3 MB
Gold Hourly	约 288	$<$ 1 MB
Gold Daily	约 12	$<$ 1 MB
Gold Top10	约 120	$<$ 1 MB
Alert	约 7	$<$ 1 MB
合计	—	**约 46 MB**

5.4　实验结果综合分析

5.4.1　情感分析准确性分析

从 Silver 表随机抽取 200 条评论进行人工标注核验，得到三分类混淆矩阵如表 5.10 所示。

表 5.10　情感三分类混淆矩阵（200 条随机样本）

	预测 positive	预测 negative	预测 neutral	行合计
真实 positive	58	3	7	68
真实 negative	4	61	8	73
真实 neutral	5	6	48	59
列合计	67	70	63	200

总体准确率 = (58+61+48)/200 = **83.5%**；正向/负向类别 Precision/Recall 均超过 90%。

中性类别的分类性能相对较弱（精确率约 76.2%），这在情感分析领域属于普遍现象：中性情感的边界模糊，即使人工标注也存在约 10% 的标注分歧。

5.4.2　热度模型有效性分析

从 Silver 表中按风险等级分层抽样，统计各等级的热度分布，如表 5.11 所示。三个风险等级的热度分布具有清晰的单调分离性，高风险评论的热度均值（18.7）约为中风险（7.2）的 2.6 倍，表明热度评分模型能够有效区分不同风险层次的评论。

表 5.11　各风险等级热度分布统计

风险等级	样本数	热度均值	热度中位数	热度最大值
`high`	134	18.7	15.4	127.3
`medium`	412	7.2	6.8	11.9
`low`	17,346	1.6	1.2	4.8

5.4.3　负面占比阈值合理性分析

在 72 小时实验周期内，系统共检测到 7 次满足告警条件的（话题, 小时）组合：其中 4 次发生于话题 T-02（社会民生事件，平均负面占比 61.3%）；2 次发生于话题 T-04（娱乐话题，平均负面占比 48.7%）；1 次发生于话题 T-01（负面占比 41.2%，经人工复核发现存在集中差评行为）。实验期内未发现明显误报案例，40% 阈值在当前实验条件下具备较好的精准度。

5.4.4　系统局限性客观讨论

在肯定上述实验结果的同时，本文对系统现存的局限性进行客观讨论：

1. **StructBERT 模型的领域适应性问题。**当前使用的通用情感分类模型在处理网络流行语、表情包文字、混合语言表达及反讽语气等非标准评论时，准确率会有所

下降。由于缺乏领域标注数据，本系统未进行领域微调，这是情感分析模块的主要改进空间。

2. **单机部署的可扩展性瓶颈。**`local[*]` 部署模式将 Spark Driver 与 Executor 运行于同一 JVM 进程，同时在 Driver 端运行 StructBERT 推理，两者存在资源竞争。在监测话题数量显著增大的场景下，单机资源可能成为瓶颈。
3. **`/api/comments` 接口的全量读取问题。**当前实现将 Silver 表全量加载为 Pandas DataFrame 后在内存中过滤，当 Silver 表数据量超过百万条时，响应时间与内存消耗将出现显著退化。
4. **爬虫合规性约束。**本系统爬虫仅采集平台公开页面内容，遵循 robots.txt 约定，并通过合理的请求间隔控制服务器压力。然而，部分平台的反爬策略持续升级，系统的长期稳定采集能力可能受到影响。

第六章　总结与展望

6.1　工作总结

本文围绕”面向多平台社交媒体的实时舆情分析与热度趋势监测”这一核心问题，完整设计并实现了一套端到端的准实时舆情监测系统。主要工作成果概括如下：

1. **多平台爬虫工程化实现。**基于 MediaCrawler 框架，采用 Chrome DevTools Protocol（CDP）接管机制实现了对微博、B 站、抖音、小红书、知乎、快手、贴吧七个主流社交媒体平台的关键词评论采集，支持可配置轮询间隔的持续采集模式，并以 JSONL 格式输出，与 Kafka 消息管道实现低耦合对接。
2. **Spark Structured Streaming 消费管道设计。**构建了基于 60 秒微批次触发的 Kafka-Spark 消费管道，实现了完整的 JSON Schema 解析、关键字段非空校验与容错处理流程，通过 Checkpoint 机制保障了异常重启后的数据不丢失。
3. **StructBERT 情感分析服务的流处理集成。**提出了懒初始化单例模式 + 批量推理 + 异常降级的三层工程化方案，将 StructBERT 中文情感分类模型嵌入 `foreachBatch` 回调，实现了批量吞吐量约 32 条/秒（`batch_size=32`）的高效推理，相较逐条推理提升约 5 倍。
4. **多维度热度评分模型。**设计并实现了综合点赞数、转发数与情感极性加成的热度评分公式，以及基于热度分数与情感标签组合的三级风险评估机制，实验验证了各风险等级热度分布的单调分离性。
5. **Delta Lake Medallion 三层湖仓架构。**在单机环境下完整实现了 Bronze（追加）—Silver（Merge 去重）—Gold（全量重算）三层 Delta Lake 数据湖架构，72 小时内处理 18,437 条评论，总存储开销约 46 MB，各端点平均响应时间低于 1.5 秒。
6. **FastAPI 数据服务与 Vue 3 可视化大屏。**设计了 9 个 RESTful 端点，实现了基于 `deltalake` Python 库的无 JVM 直读方案；前端基于 Vue 3 Composition API 与 ECharts 6 实现了支持多话题切换、双粒度趋势图、评论明细与告警面板的交互式可视化大屏，并通过 CSS 媒体查询完成了多分辨率自适应适配。

6.2 不足之处

尽管本系统在功能完整性与端到端可用性方面达到了预期设计目标，但仍存在以下值得进一步改进的局限性：

1. **情感模型的领域适应性不足。**当前系统使用的 StructBERT 通用情感分类模型在处理网络流行语、混合语言表达、反讽语气等非标准社交媒体文本时，准确率下降明显。由于缺乏高质量的领域标注语料，本系统未进行针对性微调，这是情感分析模块最核心的改进空间。
2. **单机部署的可扩展性瓶颈。**`local[*]` 部署模式下，Spark Driver 同时承载 NLP 推理与数据写入两类计算任务，资源竞争较为突出。当监测话题数量显著增加或批次数据量急剧扩大时，单机资源将成为明显瓶颈。
3. **API 全量数据加载问题。**`/api/comments` 接口当前将 Silver 表全量加载为 Pandas DataFrame 后在内存中进行过滤，当数据规模超过百万条时将出现显著的性能退化。
4. **爬虫长期稳定性约束。**各平台反爬策略持续迭代升级，CDP 接管模式在高频采集场景下存在会话失效的风险，系统的长期无人值守运行稳定性有待进一步验证与加固。

6.3 未来展望

针对上述不足，未来工作可从以下几个方向展开：

情感分析能力升级　引入 Llama-3 或 Qwen 等大语言模型（LLM）进行零样本（Zero-Shot）情感分析，或构建社交媒体领域标注数据集对 StructBERT 进行指令微调，以提升对网络流行语与隐性负面情绪的识别能力。

分布式流处理扩展　将部署模式升级为 Spark Standalone 集群或接入云厂商托管的 EMR 服务，实现 Executor 的弹性扩缩容；同时将 StructBERT 推理服务独立化为 gRPC 微服务，彻底解除推理计算与 Spark 调度之间的资源耦合。

实时存储层优化　引入 Apache Hudi 的 Merge-On-Read 增量写入机制，替代当前 Gold 层全量 Overwrite 方案，降低批次写入放大系数；对 `/api/comments` 接口改造为基于 DuckDB 的 Parquet 直查模式，规避全量加载的内存瓶颈。

舆情预测与知识图谱　在现有情感分析基础上，引入时序预测模型（如 LSTM 或 Transformer 时序模块）对热度趋势进行短期预测；结合命名实体识别（NER）构建舆情知识图谱，实现话题间关联分析。

多模态内容分析　当前系统仅分析文本评论，未来可扩展至视频弹幕、表情包文本乃至图片内容的多模态融合分析，进一步提升舆情感知的全面性。

致　谢

本文的完成离不开众多人士的关怀与支持，谨在此致以诚挚的谢意。

首先，衷心感谢指导教师**陈志**老师在本课题研究过程中给予的悉心指导与宝贵建议。陈老师以严谨的治学态度、深厚的学术积淀和对工程实践的深刻洞察，在研究方向确定、技术方案论证与论文写作规范等各个环节给予了方向性的引领与细致入微的修改意见，是本文得以顺利完成的最重要支撑。

其次，感谢天津工业大学计算机科学与技术学院为本人提供了良好的学习与研究环境，感谢大数据 2201 班同学在日常学习与项目讨论中给予的帮助与启发。

再次，本文的实现参考和使用了 Apache Spark、Delta Lake、Apache Kafka、ModelScope StructBERT、MediaCrawler、FastAPI、Vue.js、ECharts 等优秀开源项目，谨向这些项目的所有贡献者表示敬意与感谢，开源精神深刻影响了笔者对软件工程的认知。

最后，最深切的感谢献给家人多年来无怨无悔的默默支持与鼓励，是你们的理解与陪伴赋予了坚持走到今天的力量。

由于本人知识储备与实践经验尚存不足，论文中难免存在疏漏与不当之处，诚恳地请各位专家、评委和读者批评指正。

高楠

2026 年 6 月于天津

参考文献

[1] 中国互联网络信息中心（CNNIC）. 第 55 次中国互联网络发展状况统计报告[EB/OL]. 2025. https://www.cnnic.net.cn/.

[2] 陈孝和, 王飞, 张伟. 基于深度学习的微博舆情情感分析研究综述[J]. 计算机应用研究, 2023, 40(5): 1-10.

[3] TOSHNIWAL A, TANEJA S, SHUKLA A, et al. Storm@Twitter[C/OL]//Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data. ACM, 2014: 147-156. DOI: 10.1145/2588555.2595641.

[4] ZAHARIA M, DAS T, LI H, et al. Discretized streams: an efficient and fault-tolerant model for stream processing on large clusters[C]//Proceedings of USENIX HotCloud 2012. USENIX, 2012.

[5] CARBONE P, KATSIFODIMOS A, EWEN S, et al. Apache Flink: stream and batch processing in a single engine[J]. IEEE Data Engineering Bulletin, 2015, 38(4): 28-38.

[6] ARMBRUST M, DAS T, TORRES J, et al. Structured streaming: a declarative API for real-time applications in Apache Spark[C/OL]//Proceedings of SIGMOD 2018. Houston: ACM, 2018: 601-613. DOI: 10.1145/3183713.3190664.

[7] DONG Z, DONG Q. HowNet – a hybrid language and knowledge resource[C]//Proceedings of the International Conference on NLP and KE. IEEE, 2003: 820-824.

[8] HOCHREITER S, SCHMIDHUBER J. Long short-term memory[J/OL]. Neural Computation, 1997, 9(8): 1735-1780. DOI: 10.1162/neco.1997.9.8.1735.

[9] DEVLIN J, CHANG M W, LEE K, et al. BERT: pre-training of deep bidirectional transformers for language understanding[C/OL]//Proceedings of NAACL-HLT 2019. Minneapolis: Association for Computational Linguistics, 2019: 4171-4186. DOI: 10.18653/v1/N19-1423.

[10] WANG W, BI B, YAN M, et al. StructBERT: incorporating language structures into pre-training for deep language understanding[C]//Proceedings of ICLR 2020. OpenReview.net, 2020.

[11] ARMBRUST M, DAS T, SUN L, et al. Delta lake: high-performance ACID table storage over cloud object stores[C/OL]//Proceedings of VLDB 2020: Vol. 13. 2020: 3411-3424. DOI: 10.14778/3415478.3415560.

[12] Databricks. What is the medallion lakehouse architecture?[EB/OL]. 2021. https://www.databricks.com/glossary/medallion-architecture.

[13] KREPS J, NARKHEDE N, RAO J. Kafka: a distributed messaging system for log processing[Z]. 2011.

[14] RAMÍREZ S. FastAPI framework documentation[EB/OL]. 2019. https://fastapi.tiangolo.com.

[15] Vue.js Core Team. Vue 3 — the progressive JavaScript framework[EB/OL]. 2020. https://vuejs.org.

[16] Apache Software Foundation. Apache ECharts: an open source JavaScript visualization library[EB/OL]. 2021. https://echarts.apache.org.

[17] 刘勇, 韩靖云. 基于主题与多元情感融合的网络舆情动态分析方法研究[J/OL]. 竞争情报, 2021, 17(05): 10-18. DOI: 10.19442/j.cnki.ci.2021.05.004.